

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

virtual-ctor.cpp

```
15 class Sphere {  
16     public:  
17     virtual Sphere();  
18 }  
19  
20 class Ball : public Sphere {  
21     public:  
22     _____;  
23 }  
24
```

Constructors cannot be virtual

The base class is going to have all the generic code

Otherwise, it will be overwritten by the derived class

virtual-dtor.cpp

```
15 class Sphere {  
16     public:  
17         virtual ~Sphere();  
18 }  
19  
20 class Ball : public Sphere {  
21     public:  
22         ~Ball();  
23 }  
24
```

should be virtual !

Call Order – How are derived classes created?

- **base class' constructor is called first**
- **then derived class constructor is called**

Lab 3 example: drawable -> shape -> { circle, rectangle }

Call Order – How are derived classes destroyed?

- In the reverse order
- Derived class' destructor is called first
- The most base class's destructor is called last

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         void insert (T &t; unsigned index);
8
9
10
11
12
13
14
15
16
17
18     private:
19
20 };
21
22 #endif
```



```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         void insert (T &t; unsigned index);
8
9         T& remove (unsigned index);
10
11
12
13
14
15
16
17
18     private:
19
20 };
21
22 #endif
```

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         void insert (T &t; unsigned index);
8
9         T& remove (unsigned index);
10
11         bool isEmpty ( ) const;
12
13
14
15
16
17
18     private:
19
20 };
21
22 #endif
```



```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         void insert (T &t; unsigned index);
8
9         T& remove (unsigned index);
10
11         bool isEmpty() const;
12
13         unsigned length() const;
14
15         T& get (unsigned index) const;
16
17     private:
18
19
20 };
21
22 #endif
```

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         void insert (T &t; unsigned index);
8
9         T& remove (unsigned index);
10
11         bool isEmpty ( ) const;
12
13         unsigned length ( ) const;
14
15         T& get (unsigned index) const;
16
17     private:
18         List ( ); // create an empty list
19 };
20
21 #endif
```

→ depends on the implementation of the list.



List Implementations

1.

Linked list - based

2.

Array - based

List Implementations

1.

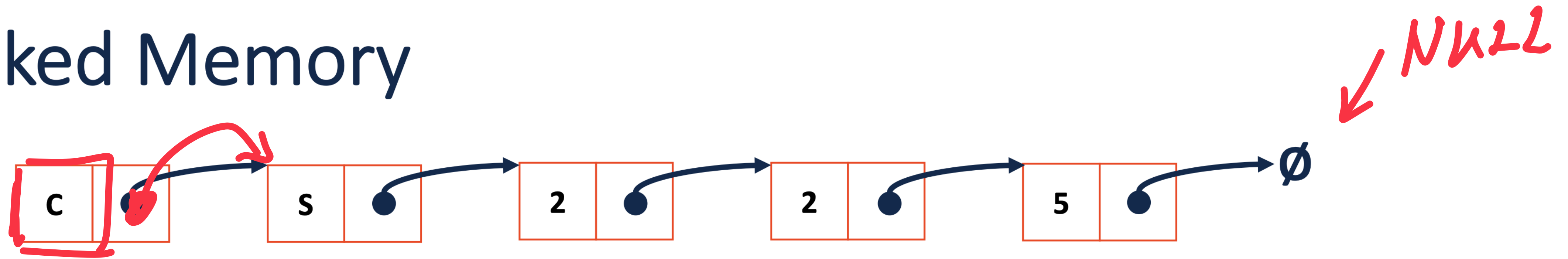
Linked list - based

2.

Array - based

ARRAY	LINKED LISTS
1. Arrays are stored in contiguous location.	1. Linked lists are not stored in contiguous location.
2. Fixed in size.	2. Dynamic in size.
3. Memory is allocated at compile time.	3. Memory is allocated at run time.
4. Uses less memory than linked lists.	4. Uses more memory because it stores both data and the address of next node.
5. Elements can be accessed easily.	5. Element accessing requires the traversal of whole linked list.
6. Insertion and deletion operation takes time.	6. Insertion and deletion operation is faster.

Linked Memory



① Every Node will be rectangle with sections

- A section with a  pointer
- Other sections are Data

② $\emptyset$ is Null.

Snippet: ListNode struct

```
1 struct ListNode {  
2     const T data;  
3     ListNode * next;  
4     ListNode(const T & data) : data(data), next(NULL) { }  
5 };
```

init list

Struct v.s. Class

- By default, all members are public
- Never contain member functions

`ListNode<int>* node = new ListNode<int>(42);`

Snippet: ListNode struct

```
1 struct ListNode {  
2     const T data;  
3     ListNode * next;  
4     ListNode(const T & data) : data(data), next(NULL) { }  
5 };
```

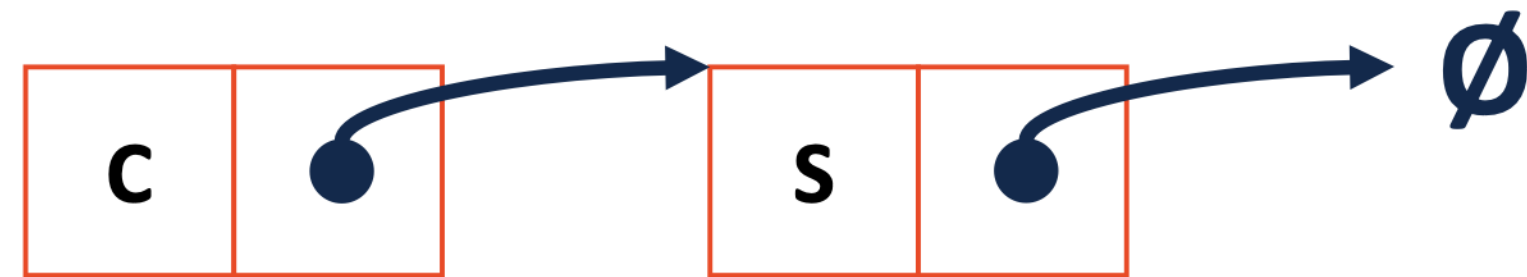
init list

Struct v.s. Class

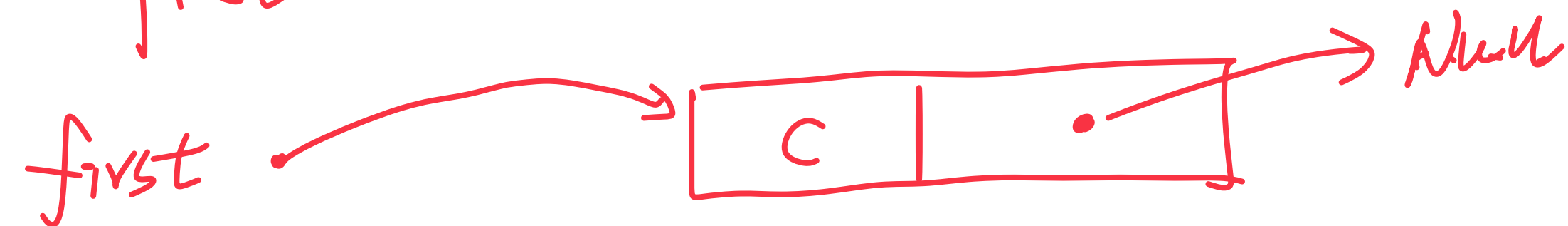
- By default, all members are public
- Never contain member functions

`ListNode<int> * node = new ListNode<int>(42);`

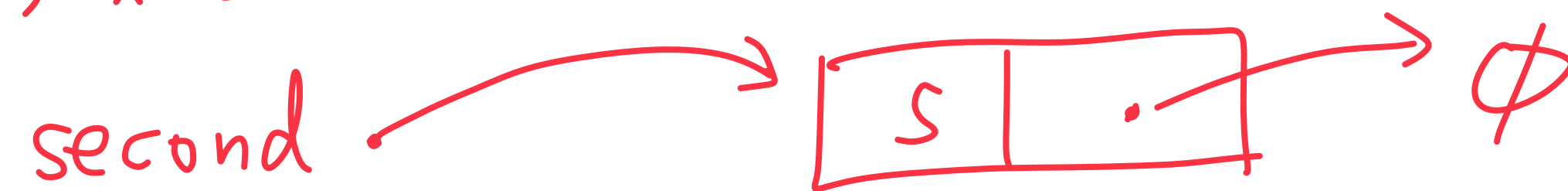
Linked Memory



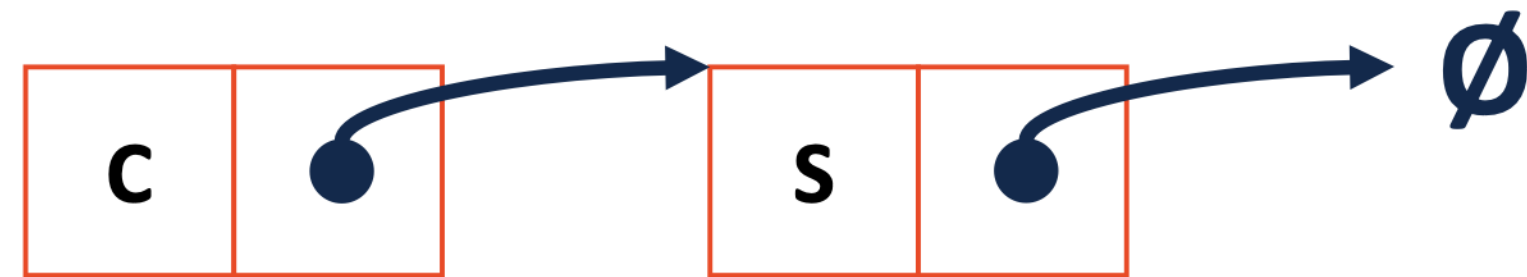
`ListNode <char> *first = new ListNode <char> ('c');`



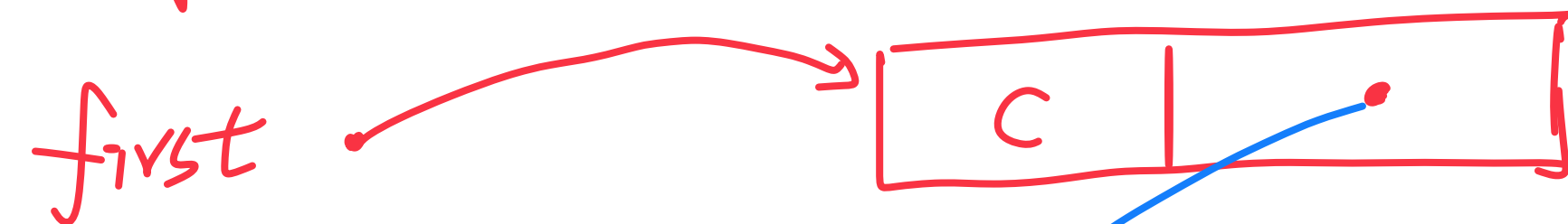
`ListNode <char> *second = new ListNode <char> ('s');`



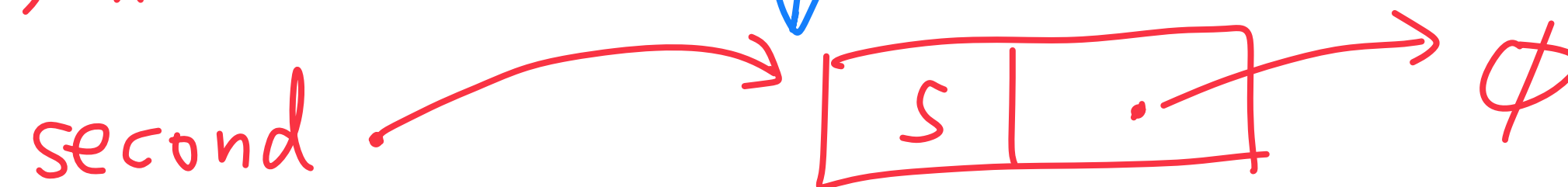
Linked Memory



`ListNode <char> *first = new ListNode <char> ('c');`

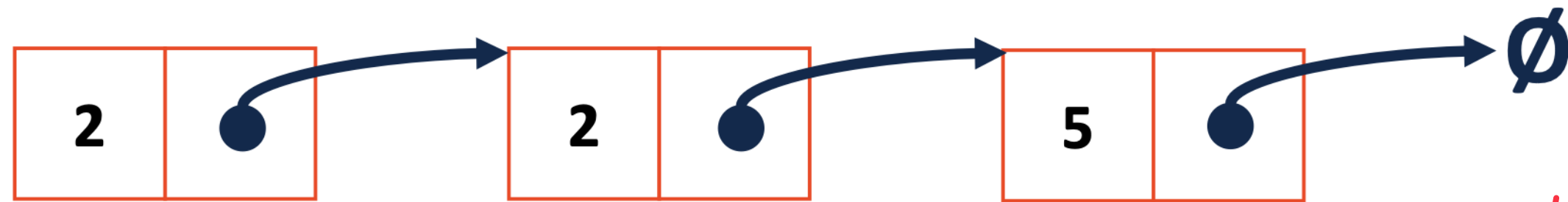


`ListNode <char> *second = new ListNode <char> ('s');`



`first->next = second;`

Linked Memory



*ListNode<char> *head = new ListNode<char>('z');*

head → ['z' | •] → $\emptyset$

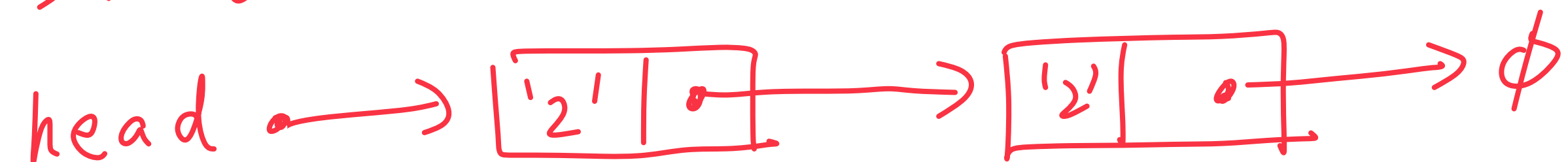
Linked Memory



`ListNode<char> * head = new ListNode<char>('z');`



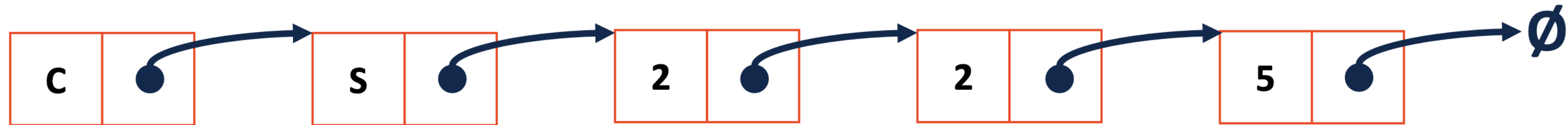
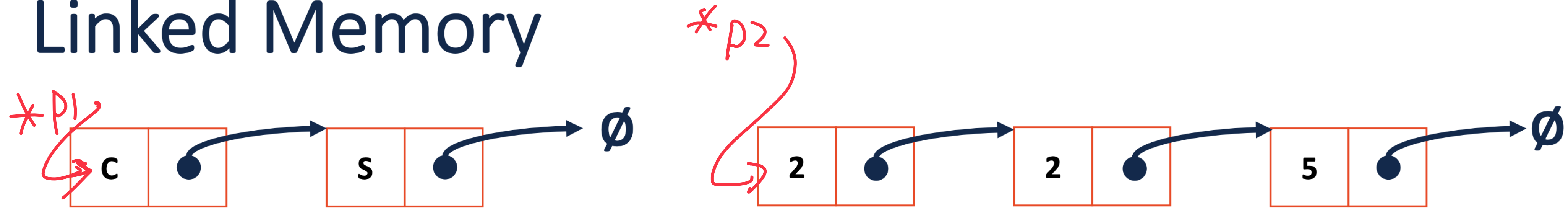
`head->next = new ListNode<char>('z');`



`head->next->next = new ListNode<char>('5');`

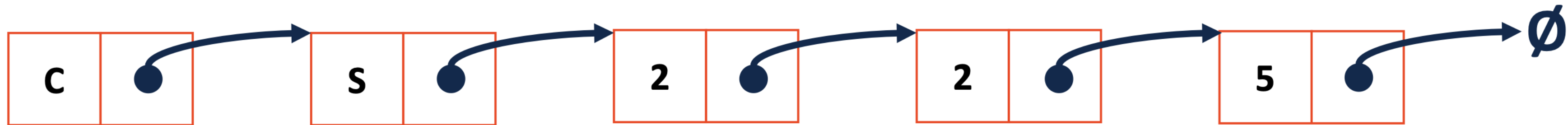
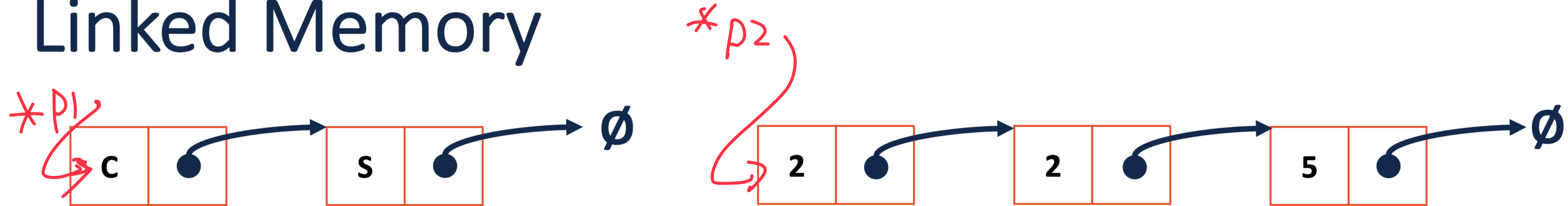


Linked Memory



$p1 \rightarrow \text{next } ??$

Linked Memory



~~$p1 \rightarrow \text{next} ??$~~

$p1 \rightarrow \text{next} \rightarrow \text{next} = p2 ;$

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15
16         ListNode <T> * head;
17
18 };
19
20 #endif
```

List.cpp

```
1
2
3 template <class T>
4 void List<T>::insertAtFront(const T& t) {
5     ListNode <T> * node = new ListNode<T>(t);
6
7     node -> [A | •] -> φ
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22 }
```


List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15
16         ListNode <T> * head;
17
18 };
19
20 #endif
```

List.cpp

```
1
2
3 template <class T>
4 void List<T>::insertAtFront(const T& t) {
5     ListNode <T> * node = new ListNode<T>(t);
6
7     node -> [A | •] -> ∅
8
9     node -> next = head
10
11     node -> [A | •] -> head
12
13
14
15
16
17
18
19
20
21
22 }
```


List.h

```

1  #ifndef LIST_H
2  #define LIST_H
3
4  template <class T>
5  class List {
6      public:
7          /* ... */
28     private:
29         struct ListNode {
30             const T data;
31             ListNode * next;
32             ListNode(const T & data) :
33                 data(data), next(nullptr) {}
34
35             ListNode <T> * head;
36
37         };
38
39     };
40
41 #endif

```

List.cpp

```

1
2
3  template <class T>
4  void List<T>::insertAtFront(const T& t) {
5      ListNode<T> * node = new ListNode<T>(t);
6
7      node -> [A | •] -> ∅
8
9
10     node -> next = head;
11
12     node -> [A | •] -> head
13
14     head = node;
15
16
17     node -> [A | •] -> ...
18     head -> [A | •]
19
20
21
22 }

```


Running Time of Linked List `insertAtFront`

Q if List is empty? # operation = ??

Running Time of Linked List **insertAtFront**

① if List is empty? # operation = ??
= 3

② if List has 1M element? # operation = ??

Running Time of Linked List **insertAtFront**

① if List is empty? # operation = ??
= 3

② if List has 1M element? # operation = ??
= 3

Constant time - $O(1)$

List.cpp

```
1
2 void List<T>::printReverse() const {
3     Any Intuition?
4
5
6 }
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
```

List.cpp

```
1
2 void List<T>::printReverse() const {
3     Any Intuition?
4     ↳ Recursion
5
6 }
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
```

Recursion: the process in which a function call itself directly or indirectly

- Tree traversal ; factorial of a number

Base case: simplest case for which the solution is trivial
↳ stop condition for Recursion

Recursion case: define the problem in terms of small problems.

★: Ensure the recursion terminates.

Recursion: the process in which a function call itself directly or indirectly

- Tree traversal ; factorial of a number

Base case: simplest case for which the solution is trivial
↳ stop condition for recursion

Recursion case: define the problem in terms of small problems.

★: Ensure the recursion terminates.

```
int fact(int n)
{
  if (n <= 1)
    return 1;
```

```
  else
    return n * fact(n-1);
}
```

if (n == 100)
 return 1;

fact(8)??

Base case not reached

⇒ stack overflow

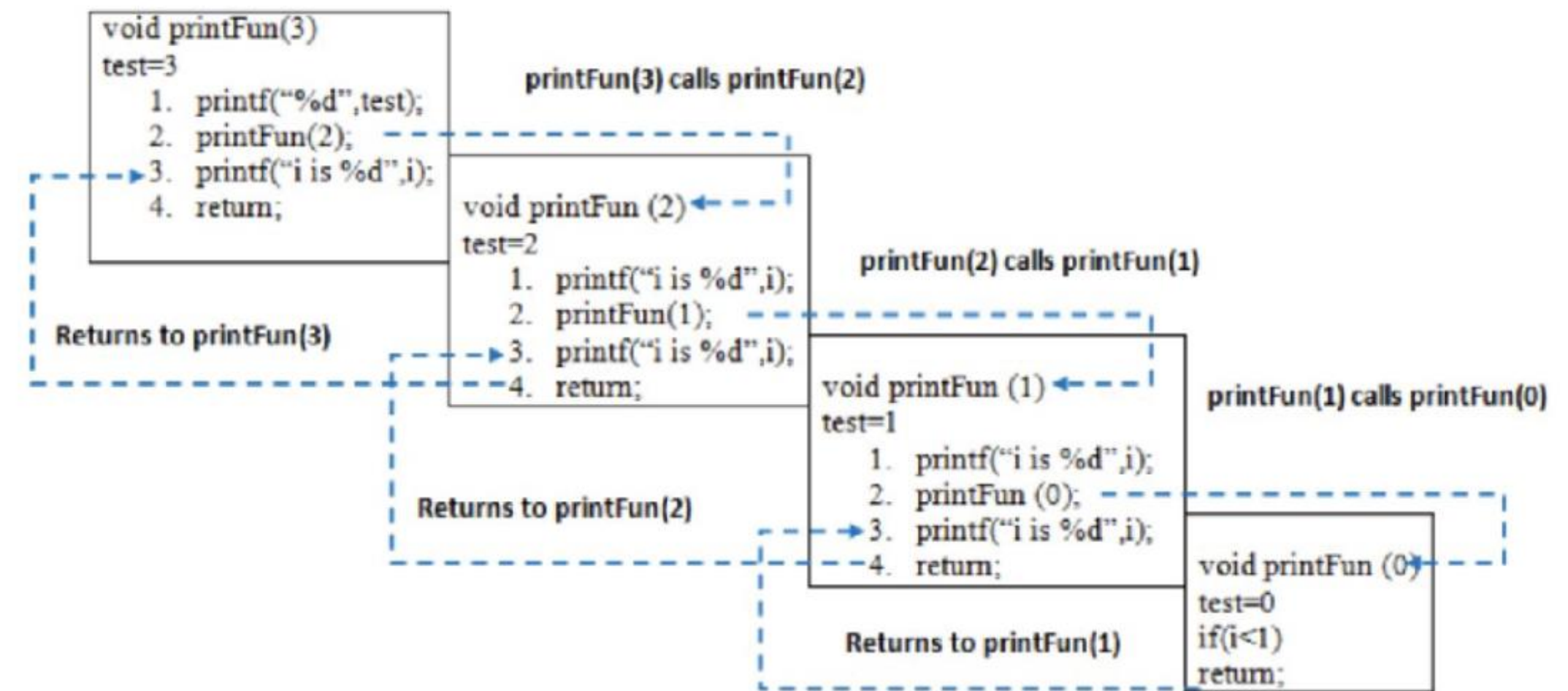

```

// A C++ program to demonstrate working of
// recursion
#include <bits/stdc++.h>
using namespace std;

void printFun(int test)
{
    if (test < 1)
        return;
    else {
        cout << test << " ";
        printFun(test - 1); // statement 2
        cout << test << " ";
        return;
    }
}

// Driver Code
int main()
{
    int test = 3;
    printFun(test);
}

```



3 2 1 1 2 3

Base case:

- End of List

$cur \rightarrow next == NULL;$

Recursion:

- Reduce the size of the list by 1

List.cpp

```
1 void List<T>::printReverse() const {
```

Any Intuition??

↳ Recursion

```
6 }
```

Base case:

- End of List

$cur \rightarrow next == NULL;$

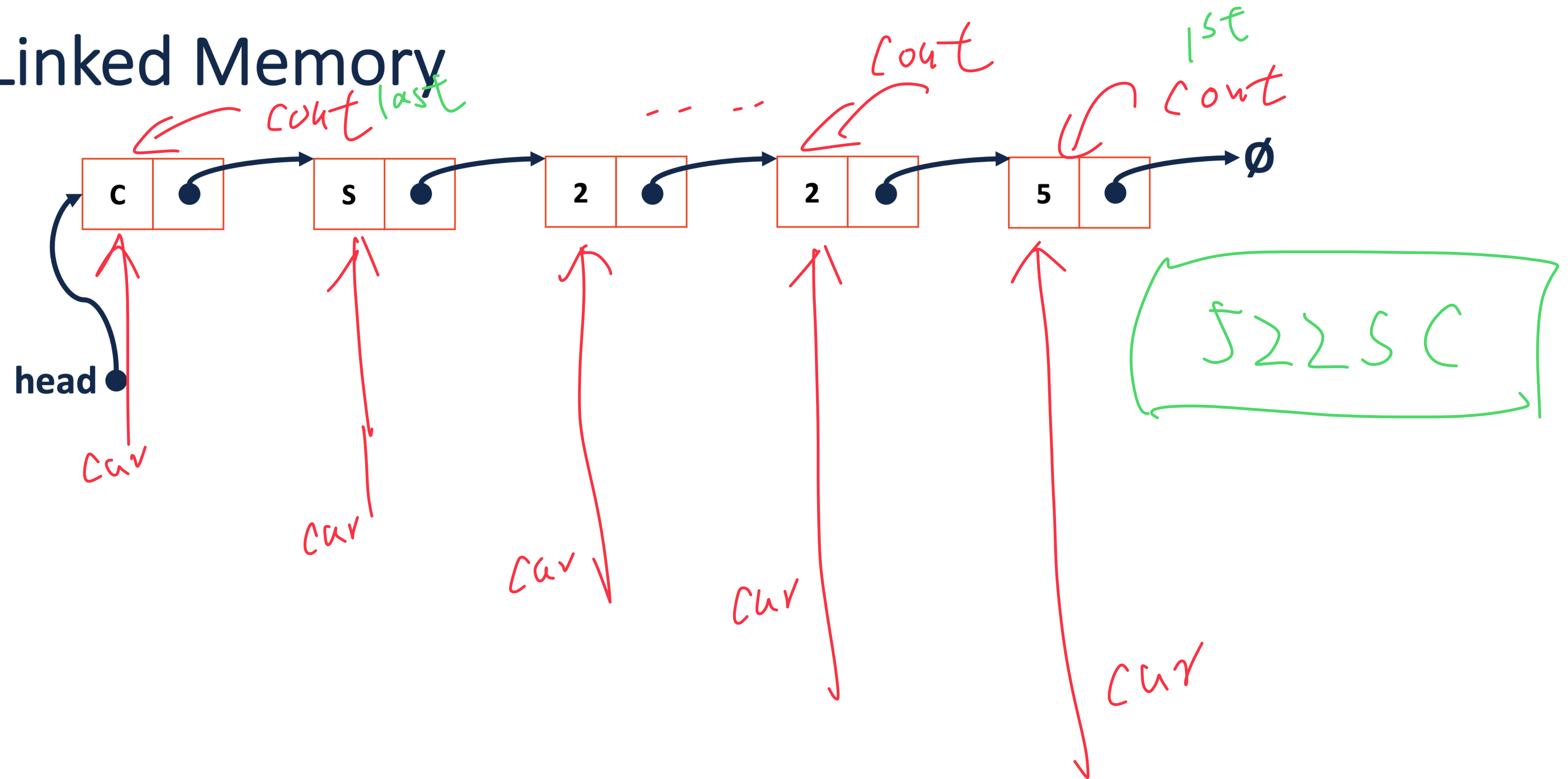
Recursion:

- Reduce the size of the list by 1

List.cpp

```
1 void List<T>::printReverse() const {  
2     Any Intuition?   
3     ↳ Recursion  
4 }  
5  
6 void List::helper(  
7     ListNode<T> *cur) {  
8     if (cur->next != NULL) {  
9         _helper(cur->next);  
10    }  
11    cout << cur->data << endl;  
12 }  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22
```

Linked Memory



Running Time of Linked List `printReverse`

$$T(n) = T(n-1) + 1$$



$O(n)$; linear time.

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15         /* ... */
16     };
17
18     /* ... */
19
20 #endif
```

List.cpp

```
1 template <class T>
2 void List<T>::insertAtFront(
3     const T& t) {
4     ListNode *node=new ListNode(t);
5     node->next = head;
6     head = node;
7 }
8
9 /* ... */
```

mainList.cpp

```
1 #include <iostream>
2 #include "List.h"
3
4 int main() {
5     List<char> *list=new List<char>;
6     list->insertAtFront('s');
7     list->insertAtFront('c');
8     list->printReverse();
9     delete list;
10    return 0;
11 }
```

template specification

g++ -std=c++1y mainList.cpp

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15         /* ... */
16     };
17
18     /* ... */
19
20 #endif
```

List.cpp

```
1 template <class T>
2 void List<T>::insertAtFront(
3     const T& t) {
4     ListNode *node=new ListNode(t);
5     node->next = head;
6     head = node;
7 }
8
9 /* ... */
```

mainList.cpp

```
1 #include <iostream>
2 #include "List.h"
3
4 int main() {
5     List<char> *list=new List<char>;
6     list->insertAtFront('s');
7     list->insertAtFront('c');
8     list->printReverse();
9     delete list;
10    return 0;
11 }
```

template specification

g++ -std=c++1y mainList.cpp

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15         /* ... */
16     };
17
18     /* ... */
19
20 #endif
```

List.cpp

```
1 template <class T>
2 void List<T>::insertAtFront(
3     const T& t) {
4     ListNode *node=new ListNode(t);
5     node->next = head;
6     head = node;
7 }
8
9 /* ... */
```

mainList.cpp

```
1 #include <iostream>
2 #include "List.h"
3
4 int main() {
5     List<char> *list=new List<char>;
6     list->insertAtFront('s');
7     list->insertAtFront('c');
8     list->printReverse();
9     delete list;
10    return 0;
11 }
```

template specification

g++ -std=c++1y mainList.cpp

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15         /* ... */
16     };
17
18     /* ... */
19
20 #endif
```

List.cpp

```
1 template <class T>
2 void List<T>::insertAtFront(
3     const T& t) {
4     ListNode *node=new ListNode(t);
5     node->next = head;
6     head = node;
7 }
8
9 /* ... */
```

mainList.cpp

```
1 #include <iostream>
2 #include "List.h"
3
4 int main() {
5     List<char> *list=new List<char>;
6     list->insertAtFront('s');
7     list->insertAtFront('c');
8     list->printReverse();
9     delete list;
10    return 0;
11 }
```

template specification

g++ -std=c++1y mainList.cpp

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7     /* ... */
8     private:
9     struct ListNode {
10         const T data;
11         ListNode * next;
12         ListNode(const T & data) :
13             data(data), next(nullptr) {}
14     };
15     /* ... */
16 };
17
18 #endif
```

List.cpp

```
1 template <class T>
2 void List<T>::insertAtFront(
3     const T& t) {
4     ListNode *node=new ListNode(t);
5     node->next = head;
6     head = node;
7 }
8
9 /* ... */
```

mainList.cpp

```
1 #include <iostream>
2 #include "List.h"
3
4 int main() {
5     List<char> *list=new List<char>;
6     list->insertAtFront('s');
7     list->insertAtFront('c');
8     list->printReverse();
9     delete list;
10    return 0;
11 }
```

template specification

g++ -std=c++1y mainList.cpp

List.h

```
1 #ifndef LIST_H
2 #define LIST_H
3
4 template <class T>
5 class List {
6     public:
7         /* ... */
8     private:
9         struct ListNode {
10             const T data;
11             ListNode * next;
12             ListNode(const T & data) :
13                 data(data), next(nullptr) {}
14         };
15         /* ... */
16     };
17
18     #include "List.cpp"
19
20 #endif
```

List.cpp

```
1 template <class T>
2 void List<T>::insertAtFront(
3     const T& t) {
4     ListNode *node=new ListNode(t);
5     node->next = head;
6     head = node;
7 }
8
9 /* ... */
```

mainList.cpp

```
1 #include <iostream>
2 #include "List.h"
3
4 int main() {
5     List<char> *list=new List<char>;
6     list->insertAtFront('s');
7     list->insertAtFront('c');
8     list->printReverse();
9     delete list;
10    return 0;
11 }
```

template specification

g++ -std=c++1y mainList.cpp

CS 225 – Things To Be Doing

lab_inheritance

Due: Sunday, March 8 (11:59pm)

MP2 Early Deadline Monday, March 9

Up to +7 Extra Credit for Early Submission